

Audit report

Vulnerability	Exploit Idea	Impact	Fix Summary	Invariant Enforced	Test Added
SQL Injection (Search)	An attacker enters command such as ' OR 1=1 -- ' into the search bar to manipulate the SQL query.	The attacker can bypass the intended filter and get all posts from database or potentially access other data.	Replaced string concatenation with parameterized SQLite queries (?) in the search query to ensure user input is treated strictly as data.	Untrusted input must not alter SQL structure.	test_sqli_login_bypass_fails – verifies that SQL injection attempts do not bypass the search filter, asserts that injecting SQL syntax into a form result in a 200 OK error pages.
IDOR / Broken Access Control (Read – Private Posts)	A logged-in user simply navigates to the feed or directly accesses /post/<id> of a private post owned by another user.	Unauthorized users can view private posts belonging to other users.	Added an authorization check verifying <code>current_user.id == post.owner_id</code> before displaying private content.	Private data must not be disclosed to unauthorized users.	Manual verification test: logged in as Bob and confirmed Alice's private posts are hidden from the feed.
IDOR / Broken Access Control (Write – Edit/Delete)	An attacker modifies the <code>post_id</code> in the request or URL when clicks "Edit" on their own post and sends an edit or delete request.	Attackers can modify or delete posts belonging to other users.	Added an ownership check (<code>if row[1] != u.id and u.role != 'admin': abort(403)</code>) in the edit and delete routes.	Users can modify only resources they own (unless admin).	test_idor_edit_blocked – attempts to edit another user's post and asserts the server returns 403 Forbidden.
Sensitive Information	An attacker	Plaintext passwords	Removed password logging from	Secrets information	Verified that login attempts recorded in admin logs no

n Disclosure (Credential Logging)	views server logs or the audit_logs database table.	of all registered users are exposed in logs.	log_event() calls in the login/register and implemented password hashing using werkzeug.security.	n must never be logged.	longer display plaintext passwords.
Broken Access Control (Admin Route)	A normal user manually navigates to /admin/logs in the browser.	Sensitive system logs are exposed to non-admin users, enabling information disclosure and potential privilege escalation.	Added role verification (if not u or u.role != 'admin': abort(403)) before loading the admin logs page.	Private data must not be disclosed to unauthorized users.	Verified that accessing the route as a non-admin returns 403 Forbidden.
Weak Session Handling (Forgeable Cookie)	An attacker can edit the browser cookie value USER_ID in Application/Storage tab in developer tools.	An attacker can impersonate another user and take over their account without knowing the password.	Replaced the raw cookie (set_cookie('USER_ID')) with Flask's built-in session mechanism using SECRET_KEY to prevent tampering.	Identity tokens must be unforgeable and integrity-protected.	Enforced across all state-changing tests implicitly. test_csrf_token_required_for_post – ensures state-changing requests require a CSRF token tied to this secure session, asserting a 403 status code if missing.

*AI Disclosure: I used an AI assistant as a learning aid during this lab. The AI helped explain the concepts behind the vulnerabilities (like CSRF and Sessions), suggested secure library APIs (like werkzeug.security**), and assisted in formatting the pytest skeleton. I manually audited the application, verified the exploits, applied the logic patches, and ran the tests myself.*